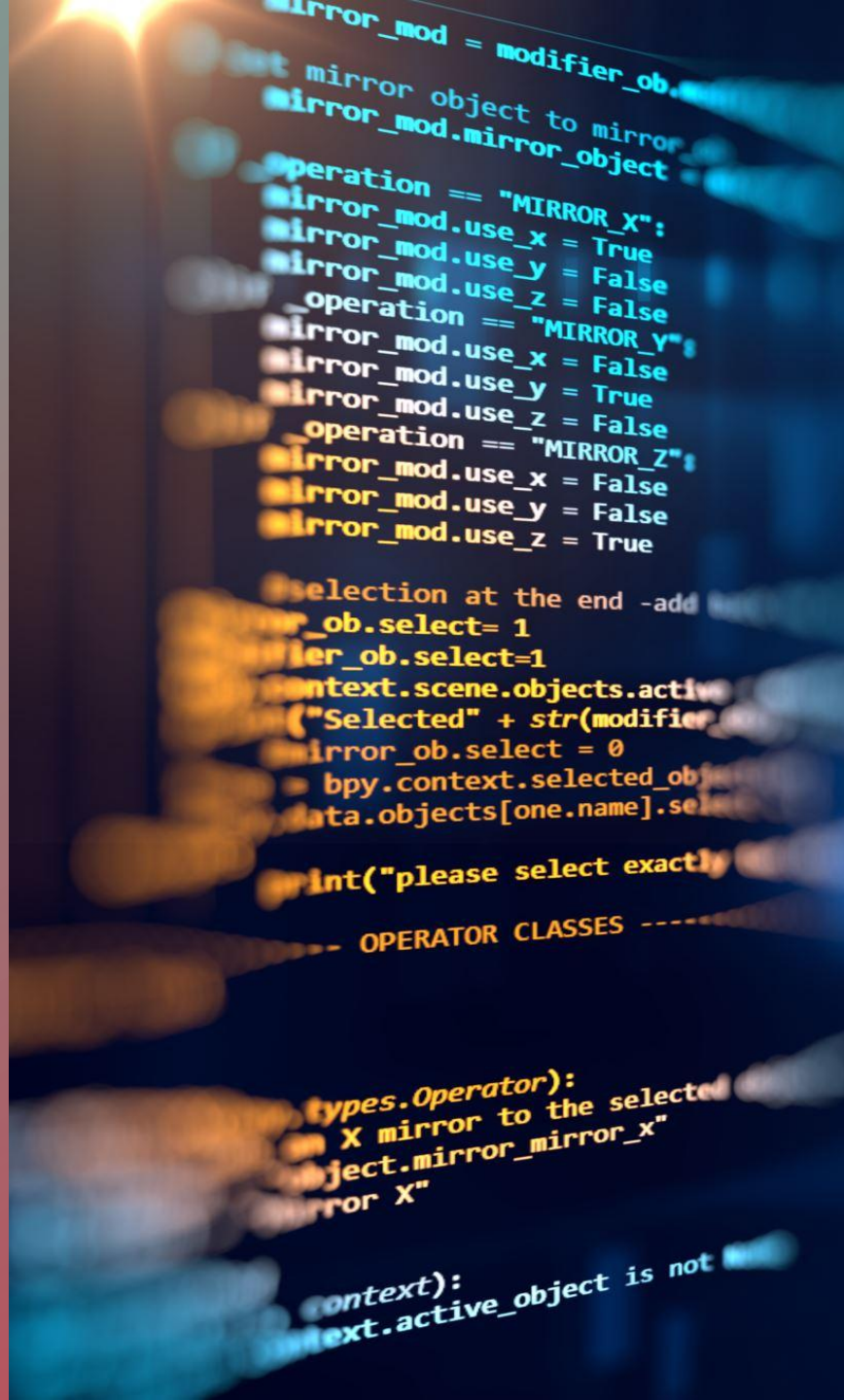
INTRODUÇÃO À ANÁLISE ASSINTÓTICA

ALGORITMOS PSEUDOPOLINOMIAIS



Análise de Algoritmos

Em 1965 Jack Edmonds introduz a ideia de Complexidade Assintótica

A ideia da “ordem de crescimento” das funções

Complexidade Assintótica

A função T(x) é chamada de Complexidade Local

A Complexidade Assintótica fornece limites para T(x)

Complexidade Assintótica

Notações

O – Big oh

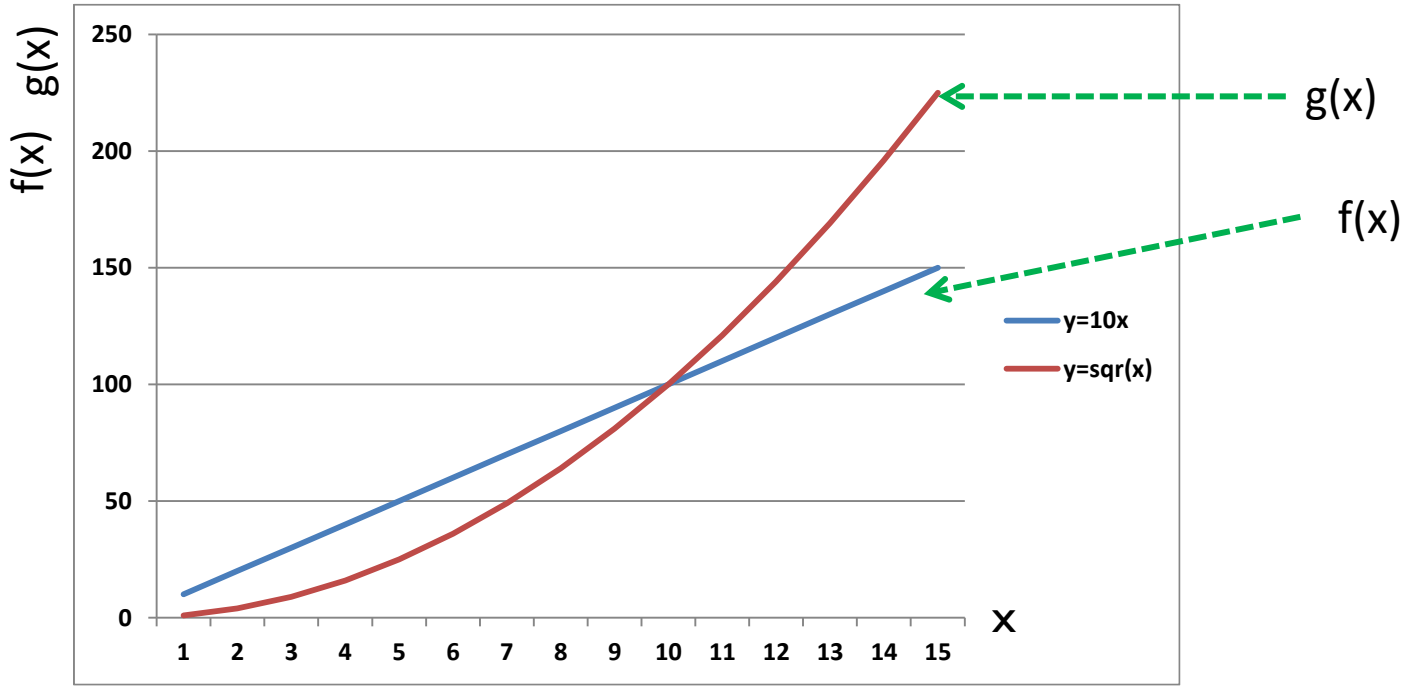
Ω

θ

Notação *O*

Considere duas funções, f(n) e g(n), não negativas, f, g: $N \rightarrow R^m$, $m \geq 0$.

Diz-se que f(n) é de ordem g(n), ou simplesmente **f(n) é *O*(g(n))** se o crescimento de f(n) é, no máximo, tão rápido quanto o crescimento de g(n).



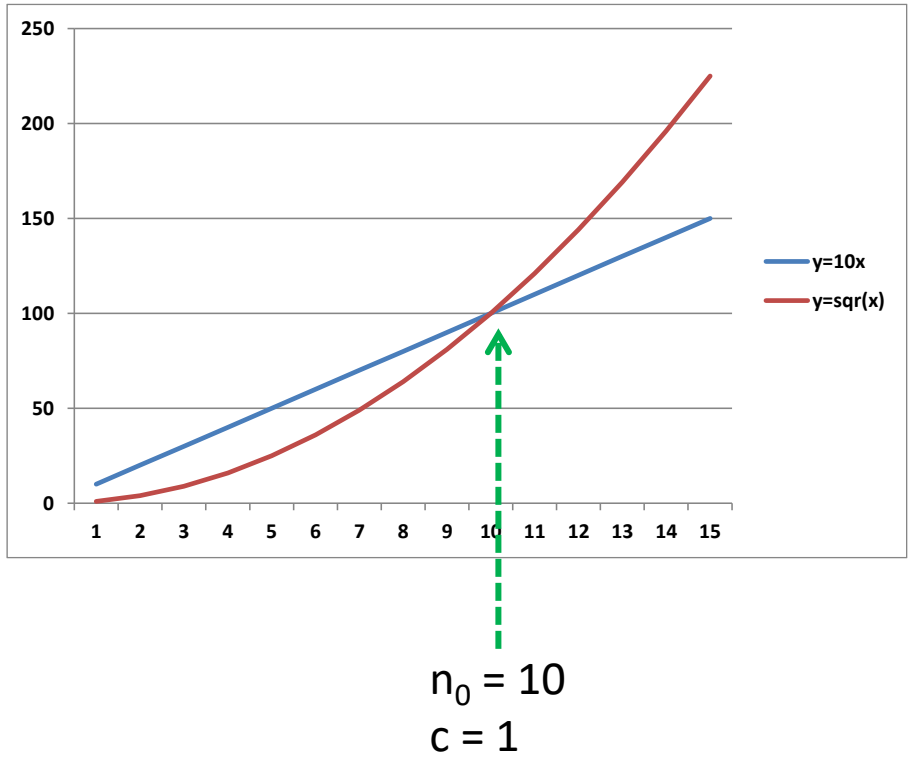
Notação *O*

f(n) é limitada superiormente por um múltiplo real positivo de g(n) para valores grandes de n.

Diz-se que **f(n) é *O*(g(n))** quando:

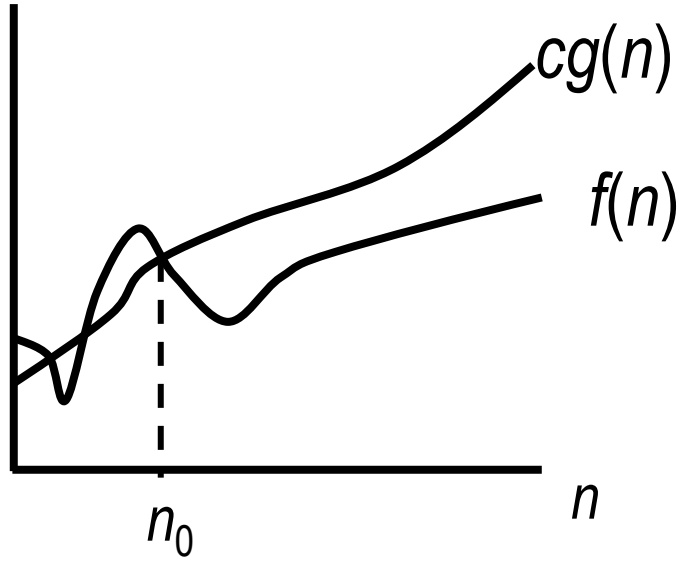
existe uma constante real positiva *c* e uma constante inteira positiva n_0 tais que

$f(n) \leq cg(n)$, para todo valor de $n > n_0$.



Notação *O*

Esta definição equivale a dizer que $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ existe e é finito.



$f(n)=O(g(n))$

Exemplos

$$\lim_{n\to\infty} \frac{\sqrt{n}}{n} = 0 \longrightarrow \sqrt{n} = O(n)$$

$$\lim_{n\to\infty} \frac{n}{\sqrt{n}} = \infty \longrightarrow n \text{ não é } O(\sqrt{n})$$

$$\lim_{n\to\infty} \frac{n}{2n} = \frac{1}{2} \longrightarrow n = O(2n)$$

$$\lim_{n\to\infty} \frac{2n}{n} = 2 \longrightarrow 2n = O(n)$$

Exemplos

$7n-2$

$7n-2$ é $O(n)$

Exige-se $c > 0$ e $n_0 \geq 1$ tal que $7n-2 \leq cn$ para $n \geq n_0$

O que é verdade para, por exemplo, $c = 7$ e $n_0 = 1$

$3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ é $O(n^3)$

Exige-se $c > 0$ e $n_0 \geq 1$ tal que $3n^3 + 20n^2 + 5 \leq cn^3$ para $n \geq n_0$

O que é verdade para, por exemplo, $c = 4$ e $n_0 = 21$

Exemplos

$3 \log n + 5$

$3 \log n + 5$ é $O(\log n)$

Exige-se $c > 0$ e $n_0 \geq 1$ tal que $3 \log n + 5 \leq c \log n$ para $n \geq n_0$

O que é verdade para, por exemplo, $c = 8$ e $n_0 = 2$

Regra da Soma

Se um algoritmo A se divide em duas partes independentes, A_1 e A_2, com complexidades dadas por $T_1(n)$ e $T_2(n)$ de *ordem* $O(f(n))$ e $O(g(n))$, respectivamente, então

$$T(n) = T_1(n) + T_2(n)$$

e

A será de ordem $O(\max \{f(n), g(n)\})$.

Regra da Soma

Se um algoritmo A se divide em duas partes independentes, A_1 e A_2, com complexidades dadas por $T_1(n)$ e $T_2(n)$ de *ordem* $O(f(n))$ e $O(g(n))$, respectivamente, então $T(n) = T_1(n) + T_2(n)$ e A será de ordem $O(max\{ f(n),g(n)\})$.

```
Algoritmo A (n)
    para i ← 1 até n
        imprima(i)
    para i ← 1 até n
        para j ← 1 até n
            imprima(i×j)
```

A_1 $T_1(n)$ é $O(n)$

A_2 $T_2(n)$ é $O(n^2)$

$T(n)$ é $O(n^2)$

Regra do Produto

Se um algoritmo A tem duas partes interdependentes, A_1 e A_2, com complexidades dadas por $T_1(n)$ e $T_2(n)$ de *ordem* $O(f(n))$ e $O(g(n))$, respectivamente, então

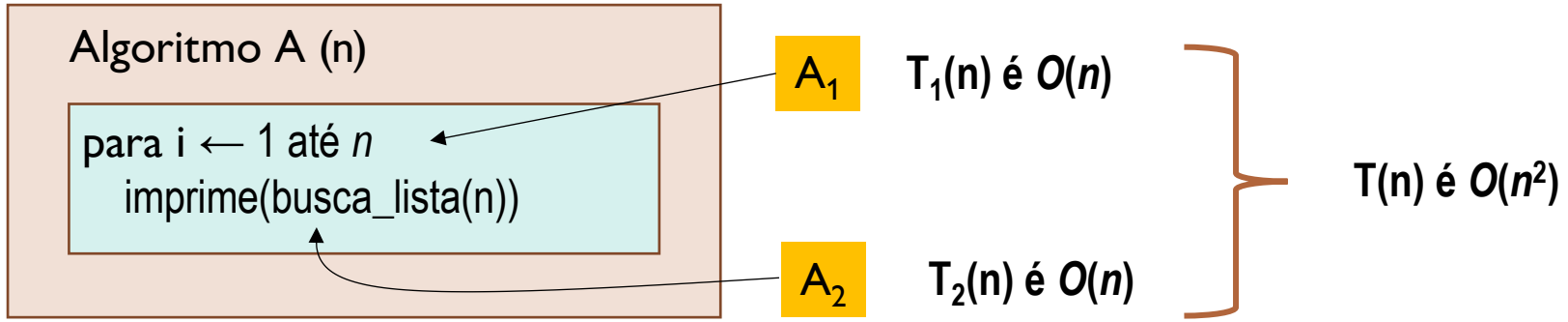
$$T(n) = T_1(n) \times T_2(n)$$

e

T(n) é $O(f(n) g(n))$.

Regra do Produto

Se um algoritmo A tem duas partes interdependentes, A_1 e A_2, com complexidades dadas por $T_1(n)$ e $T_2(n)$ de *ordem* $O(f(n))$ e $O(g(n))$, respectivamente, então $T(n) = T_1(n) \times T_2(n)$ e $T(n)$ é $O(f(n)g(n))$.



Resumo: Notação *O*

1. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} \in \Re_+^*$ então $f(n) \in O(g(n))$ e $g(n) \in O(f(n))$

2. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ então $f(n) \in O(g(n))$ e $g(n) \notin O(f(n))$

3. Se $\lim_{n\to\infty} \frac{f(n)}{g(n)} = +\infty$ então $f(n) \notin O(g(n))$ e $g(n) \in O(f(n))$

Notação Ω

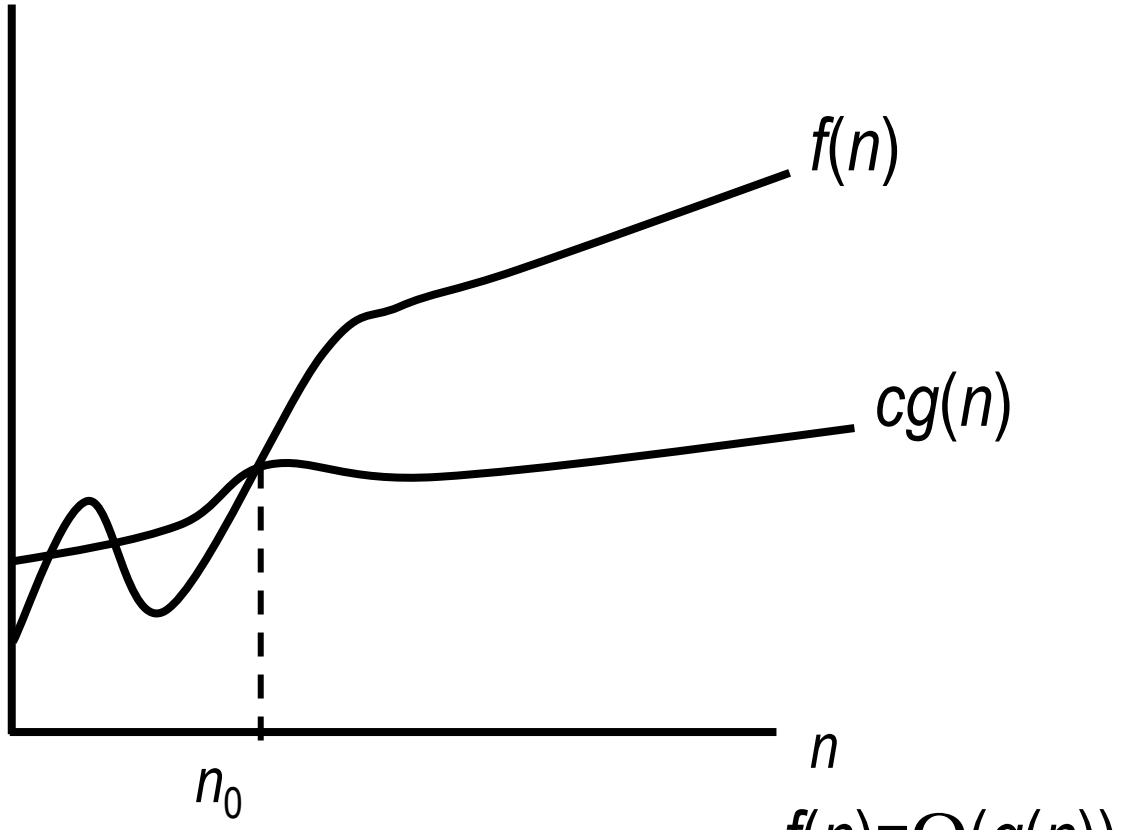
Diz-se que **f(n) é Ω(g(n))** quando:

existe uma constante real positiva *d* e um valor inteiro positivo n_0 tais que

$f(n) \geq d\ g(n)$, para todo valor de $n > n_0$.

A regra da dualidade estabelece que: **g(n)** é **Ω(f(n))** se e somente se **f(n) é *O*(g(n))**, uma vez que **$g(n) \geq df(n)$** se e somente se **$f(n) \leq 1/d\ g(n)$.**

Assintótica Ω



$f(n)=\Omega(g(n))$

Exemplo

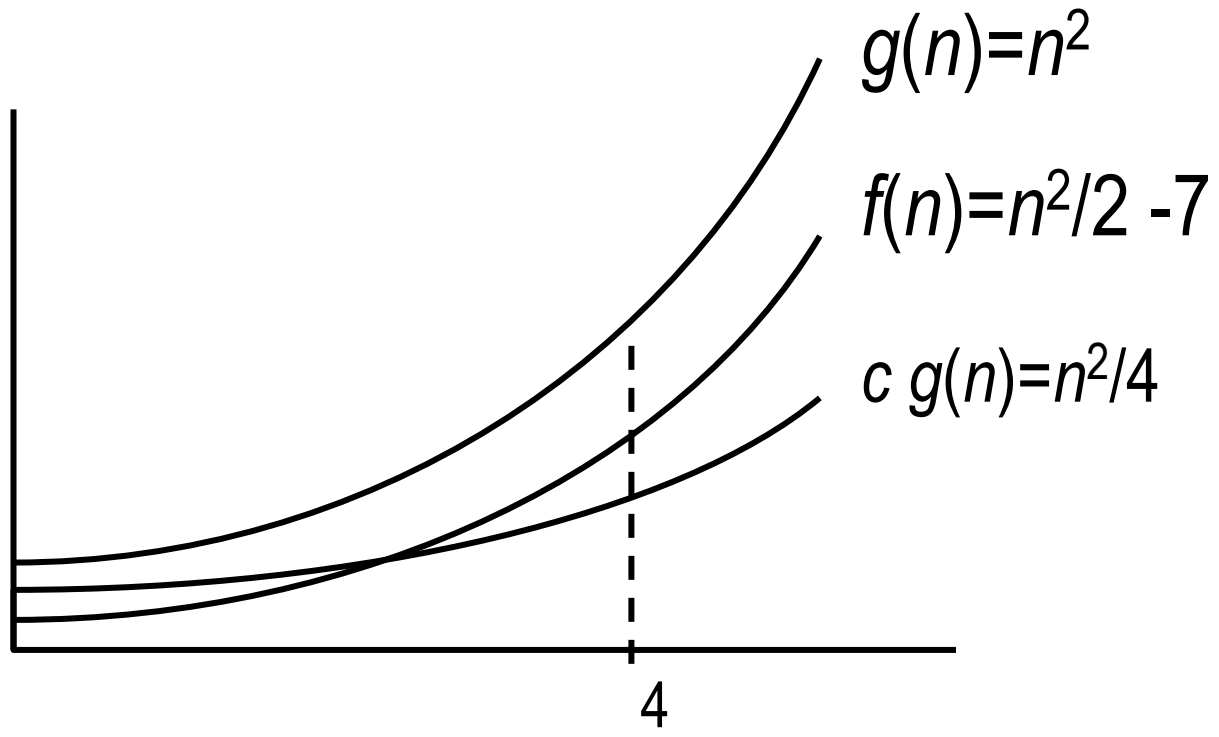
$g(n)/4 = n^2/4$

$= n^2/2 - n^2/4$

$\leq n^2/2 - 9$ para todo $n \geq 6$

$< n^2/2 - 7$

Então, $f(n) = \Omega(g(n)) = \Omega(n^2)$



Exemplo

$f(n) = 100n + 5$ é $\Omega(g(n))$, $g(n) = n^2$?

Para encontrar c, n_0 tal que $0 \leq cn^2 \leq 100n + 5$

$100n + 5 \leq 100n + 5n$ (para $n \geq 1$) $= 105n$

Como n é positivo

$cn^2 \leq 105n \quad \Rightarrow cn \leq 105 \Rightarrow n \leq 105/c$

Contudo n não pode ser menor que uma constante!

Notação Θ

Diz-se que que **f(n)** é **Θ (g(n))**, ou que **f(n)** é da ordem exata de **g(n),** se **f(n)** é tanto ***O*(g(n))** como **Ω(g(n))**.

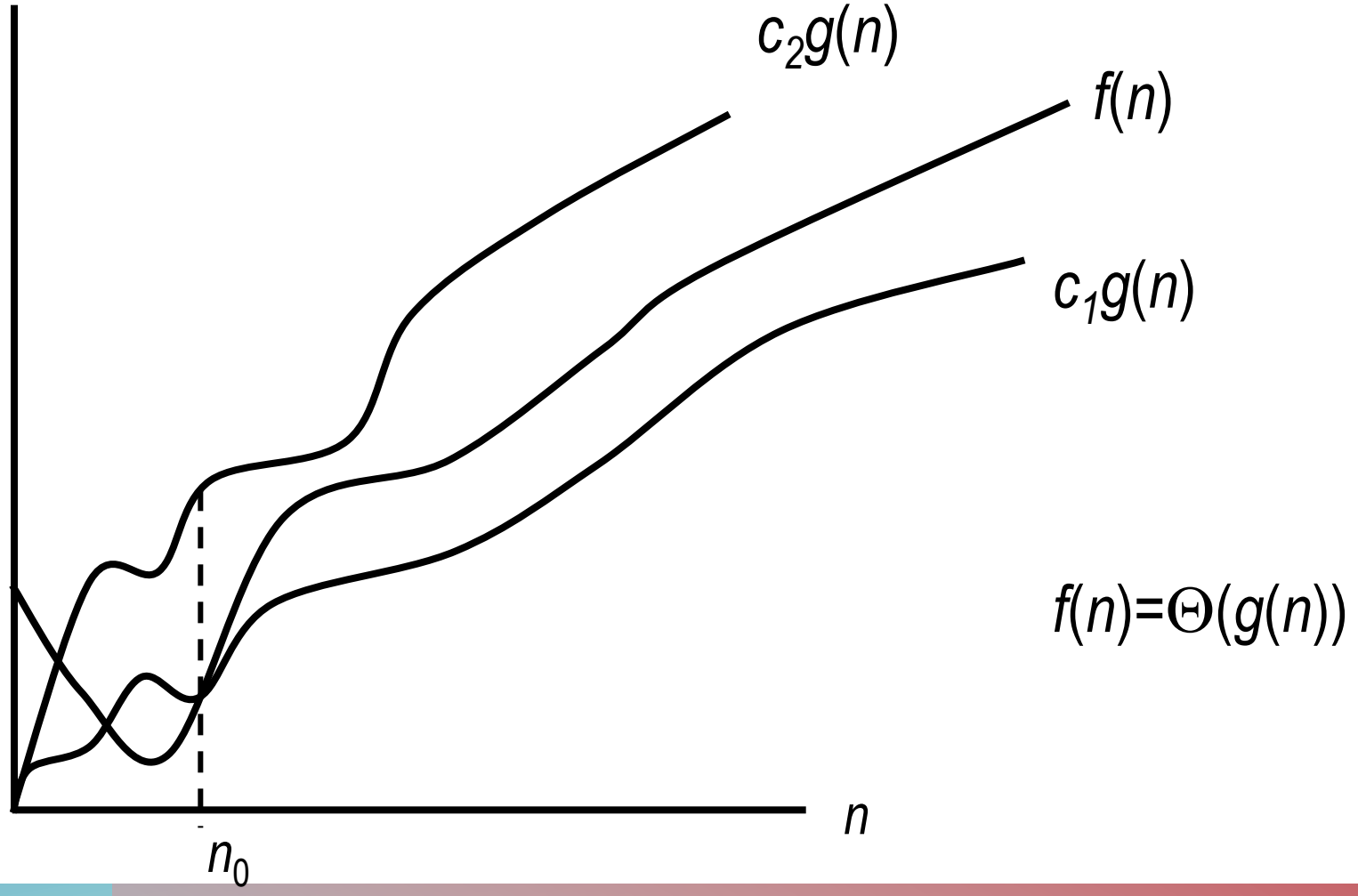
Formalmente, **Θ(g(n)) = O (g(n)) $\cap$ Ω(g(n))**, o que equivale dizer que existem constantes reais positivas **c** e **d**, bem como valores inteiros positivos n_1 e n_2 tais que:

$$f(n) \leq cg(n), \text{ para todo } n > n_1$$

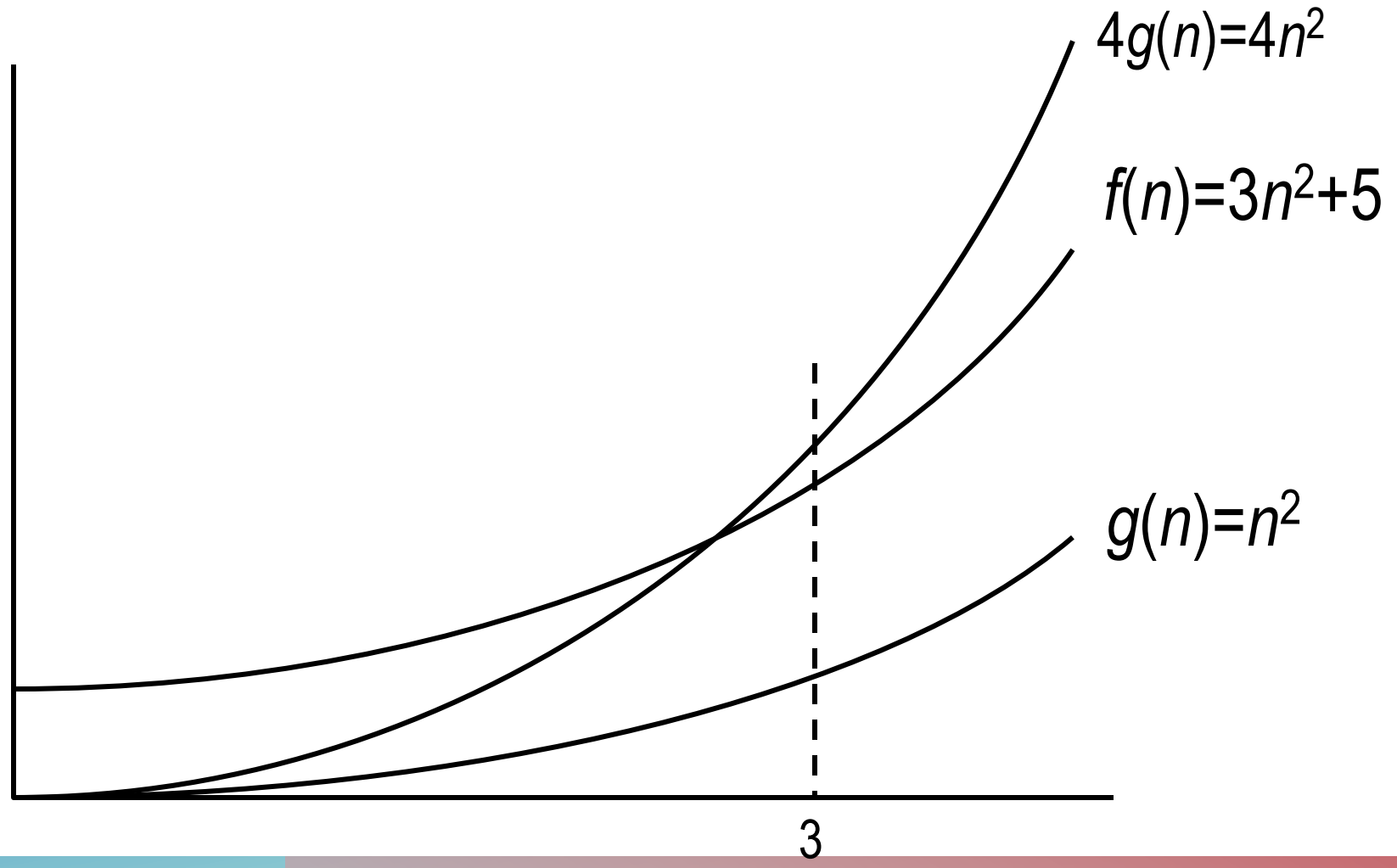
e

$$f(n) \geq dg(n), \text{ para todo } n > n_2$$

Assintótica Θ



Exemplo



Notação o

Diz-se que **f(n) é *o*(g(n))** quando:

existe uma constante real positiva *c* e uma constante inteira positiva n_0 tais que

$$f(n) < cg(n), \text{ para todo valor de } n > n_0$$

Notação o

Pode-se dizer ainda que f(n) é *o*(g(n)), $n \to \infty$, quando, para *qualquer* constante positiva ε, existe uma constante n_0 tal que

$$|f(n)| \leq \varepsilon\, |g(n)|, \text{ para } n \geq n_0$$

Uma definição equivalente é:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

Notação ω

Diz-se que **f(n) é ω(g(n))** quando:

existe uma constante real positiva *d* e uma constante inteira positiva n_0 tais que

$$f(n) > dg(n), \text{ para todo valor de } n > n_0$$

Notação ω

Pode-se dizer ainda que f(n) é ω(g(n)), $n \to \infty$, quando, para *qualquer* constante positiva ε, existe uma constante n_0 tal que

$|f(n)| \geq \varepsilon |g(n)|$, para **$n \geq n_0$**

Uma definição equivalente é:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = +\infty$$

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Entrada
a e n: inteiros

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Entrada
a e n: inteiros

O que faz este algoritmo?

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Entrada
a e n: inteiros

O que faz este algoritmo?

Calcula a^n.

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Qual a complexidade de pior caso em notação assintótica?

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Qual a complexidade de pior caso em notação assintótica?

Operação que mais se repete

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Qual a complexidade de pior caso em notação assintótica?

Operações em tempo constante

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Qual a complexidade de pior caso em notação assintótica?

Cada iteração k é dividido por 2

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Qual a complexidade de pior caso em notação assintótica?

$O(\log n)$ testes

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Melhor caso: mesmo número de operações

$\Omega(\log n)$ testes

Exercício

```
Expo_Rap (a, n)
z ← 1
t ←  a
k ← n
Enquanto k>0
      se (k é impar) então z ← z * t
       k ← ⌊k/2⌋
       se k ≠ 0 então t ← t * t
Escrever ( z )
```

Pior caso: $O(\log n)$ testes

Melhor caso: $\Omega(\log n)$ testes

Complexidade $\theta(\log n)$

Ordenação por Inserção

```
1.  i = 2
2.  faça
3.      x = A[i]
4.      j = i - 1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] = A[j]
8.          j = j – 1
9.      faça
10.     A[j+1] = x
11.     i = i + 1
12. enquanto (i <= n)
```

procedimento inserção (A[1..*n*])

Pior caso: $O(n^2)$

Melhor caso: $\Omega(n)$

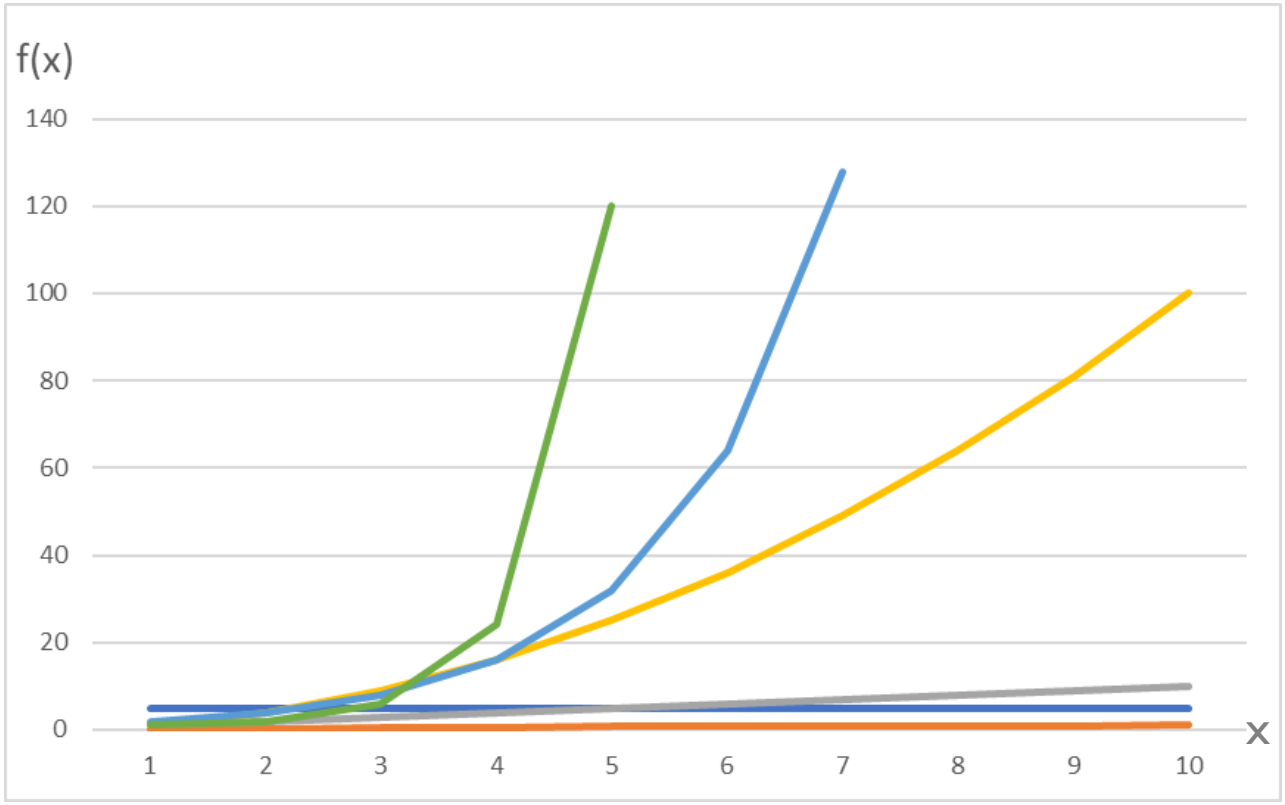
Complexidade $O(n^2)$

Funções

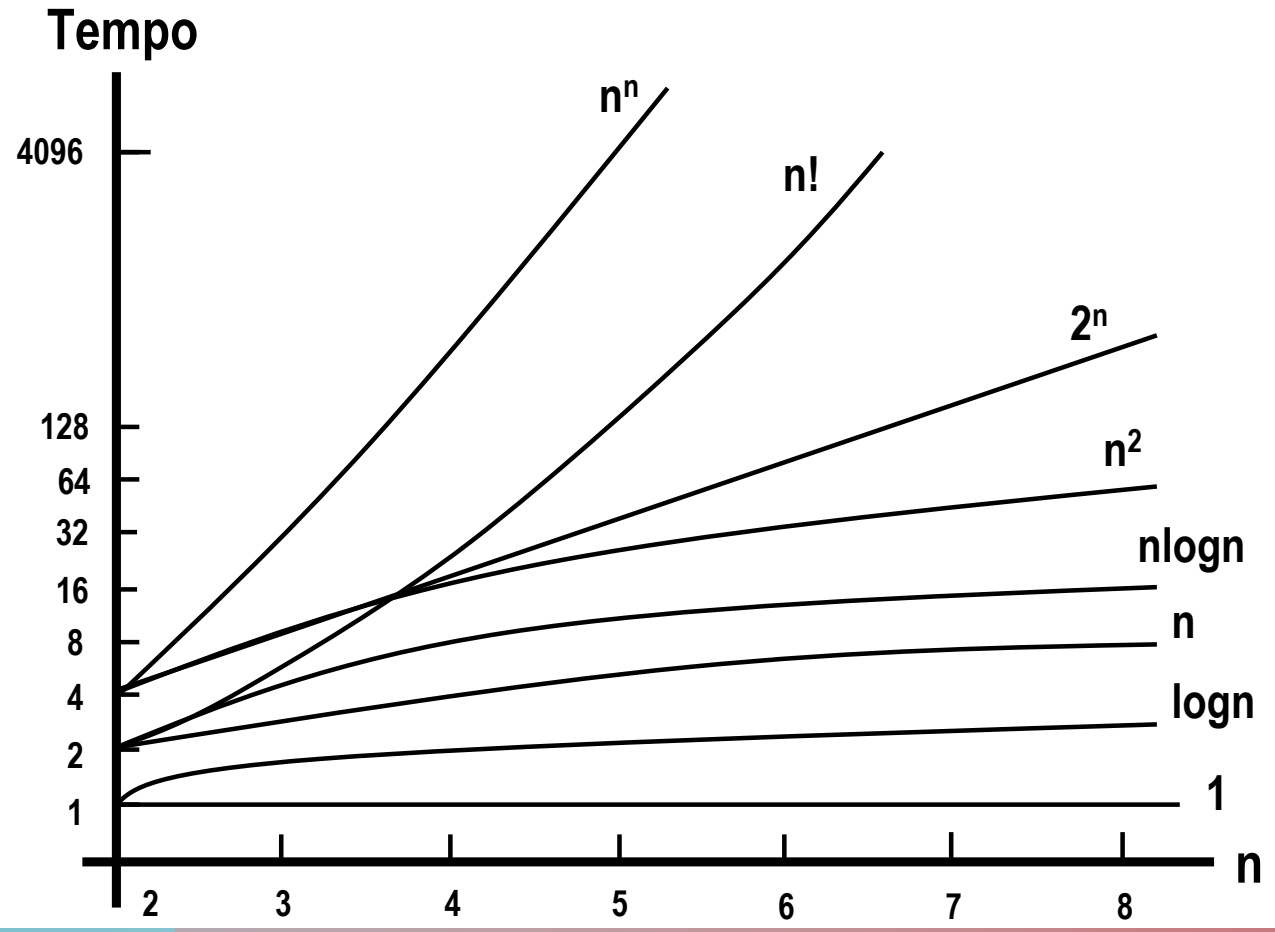
Função	Nome	Função	Nome
C	Constante	n^2	Quadrada
Log n	Logarítmica	n^3	Cúbica
Log^2 n	Log quadrada	2^n	Exponencial
n	Linear	n!	Fatorial

Funções

x	f(x) = 5	f(x) = logx	f(x) = x	$f(x) = x^2$	$f(x)=2^x$	f(x) = x!
1	5	0	1	1	2	1
2	5	0,30103	2	4	4	2
3	5	0,477121	3	9	8	6
4	5	0,60206	4	16	16	24
5	5	0,69897	5	25	32	120
6	5	0,778151	6	36	64	720
7	5	0,845098	7	49	128	5040
8	5	0,90309	8	64	256	40320
9	5	0,954243	9	81	512	362880
10	5	1	10	100	1024	3628800



Crescimento



Crescimento

Quais das duas funções cresce mais rápido?

$x^{0,01}$ ou *log x* ?

$$\lim_{x \to \infty} \frac{x^{0,01}}{\log x} \rightarrow \frac{\infty}{\infty}$$

Crescimento

Aplicando l´Hôpital

$$\frac{0{,}01\,x^{-0{,}99}}{(1/x)} = 0{,}01\,x^{0{,}01}$$



Guillaume l´Hôpital

Consequentemente, o limite tende para o infinito e $x^{0,01}$ não é O ($log\ x$).

Por outro lado, a razão inversa tende para zero, então $log\ x$ é O ($x^{0,01}$) e o ($x^{0,01}$).

Crescimento

Melhor



Pior

- $O(1)$ constante
- $O(\log n)$ logarítmica
- $O(n)$ linear
- $O(n \log n)$ log linear
- $O(n^2)$ quadrática
- $O(n^3)$ cúbica
- $O(2^n)$ exponencial

Exemplo

Considere 5 algoritmos A_1...A_5 que resolvem um mesmo problema com um tempo de 1 ms em cada operação

$T_k(n)$é a complexidade do algoritmo *K* na entrada *n*

n	A_1 $T(n)=n$	A_2 $T(n)=n\log n$	A_3 $T(n)=n^2$	A_4 $T(n)=n^3$	A_5 $T(n)=2^n$
16	**0,016s**	**0,064s**	**0,256s**	**4s**	**1m4s**
32	**0,032s**	**0,16s**	**1s**	**33s**	**46d**
512	**0,512s**	**9s**	**4m22s**	**1d13h**	**10^{137} Séculos**

Exemplo

Considere 5 algoritmos A_1...A_5 que resolvem um mesmo problema com um tempo de 1 ps em cada operação

$T_k(n)$é a complexidade do algoritmo *K* na entrada *n*

n	A_1 $T(n)=n$	A_2 $T(n)=n\log n$	A_3 $T(n)=n^2$	A_4 $T(n)=n^3$	A_5 $T(n)=2^n$
16	1,6E-11s	6,4E-11s	2,56E-10s	4E-09s	6,5E-08s
32	3,2E-11s	1,6E-10s	1 E-09s	3,3E-08s	4,3E-03s
512	5,12E-11s	4,6E-09s	2,6E-07s	1,3E-04s	10^{132} Séculos

Algumas fórmulas úteis

Série Aritmética	Série Geométrica	Série Harmônica
$\sum_{i=0}^{n} i = \frac{n(n+1)}{2} \equiv O(n^2)$	$\sum_{i=0}^{n} x^i = \frac{x^{n+1}-1}{x\text{-}1} \equiv O(x^n)$	$\sum_{i=1}^{n} \frac{1}{k} = \ln n + O(1) \equiv O(\ln n)$
$\sum_{i=0}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \equiv O\left(\frac{n^3}{3}\right)$	$\sum_{i=1}^{n} \frac{i}{2^i} \leq 2$	$\sum_{i=1}^{n} \frac{1}{k} = \ln n + O(1) \equiv O(\ln n)$

$$n! = \sqrt{2\pi\, n}\left(\frac{n}{e}\right)^n\left(1+\Theta\left(\frac{1}{n}\right)\right)$$ Aproximação de Stirling

Algumas fórmulas úteis

$$\sum_{i=1}^{n} \log i$$

$$S_n = \log 1 + \log 2 + \log 3 + \ldots + \log n$$

$$S_n = \log(1.2.3 \ldots n)$$

$$S_n = \log(n!)$$

$$\log(n!) = \Theta(n \log n)$$

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

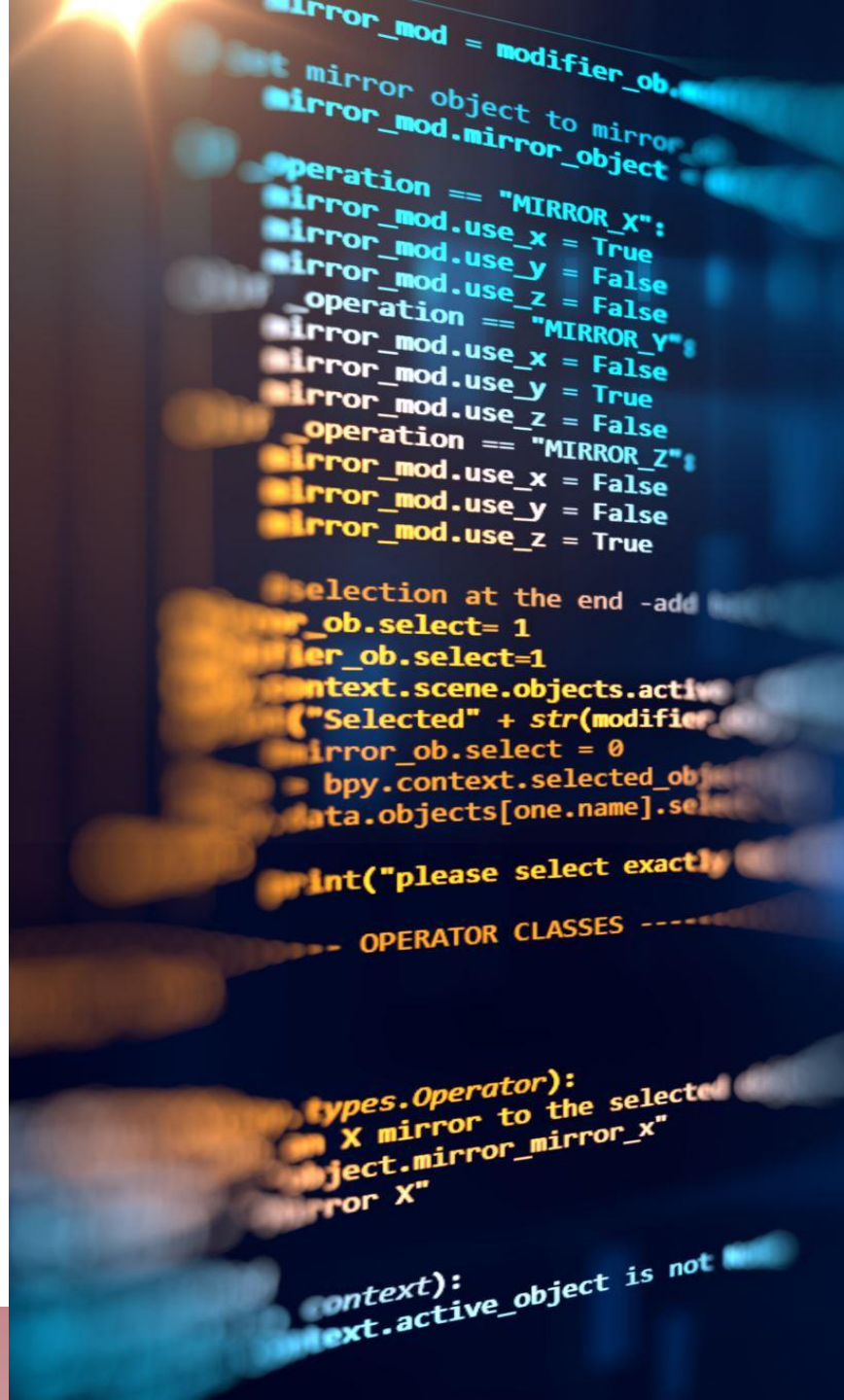
Algumas fórmulas úteis

$$\sum_{i=1}^{n-1} \frac{1}{i(i+1)} = \frac{1}{i(i+1)} = \frac{1}{i} - \frac{1}{i+1}$$

$$S_n = \sum_{i=1}^{n-1} \left(\frac{1}{i} - \frac{1}{i+1} \right) = \left(1 - \frac{1}{2}\right) + \left(\frac{1}{2} - \frac{1}{3}\right) + \left(\frac{1}{3} - \frac{1}{4}\right) + \ldots + \left(\frac{1}{n-1} - \frac{1}{n}\right)$$

$$S_n = 1 - \frac{1}{n}$$

ALGORITMO PSEUDOPOLINOMIAL



Algoritmos Pseudopolinomiais

A complexidade de um algoritmo é dada como função do *tamanho da sua entrada*.

Um número *k* codificado em *binário* é representado por $\lfloor \log_2 k \rfloor + 1$ dígitos.

Esta codificação é tradicionalmente aceita porque não altera a grandeza da complexidade do algoritmo, ou seja, se um algoritmo é polinomial ou exponencial, ele continua sendo polinomial ou exponencial, mesmo que a base da codificação seja alterada.

Algoritmos Pseudopolinomiais

O mesmo pode não ocorrer se a codificação for unária.

Definição:
Um algoritmo A é dito de complexidade pseudopolinomial se sua complexidade é polinomial para o tamanho da entrada, quando esta é codificada em unário.

Isto é, a complexidade é polinomial no valor da entrada (e não no tamanho da entrada em codificação binária).

Algoritmos Pseudopolinomiais

Problema: Determinar se um número *n* é primo

O laço itera $\sqrt{n} - 1$ vezes.

Tamanho da entrada X número de passos ?

Algoritmos Pseudopolinomiais

Problema: Determinar se um número *n* é primo

Primo(*n*)

para i ← 2 até $\sqrt{n}$ fazer
  se (*n* mod i) = 0 então
    retorne("não primo")
retorne("primo")

O laço itera $\sqrt{n} - 1$ vezes.

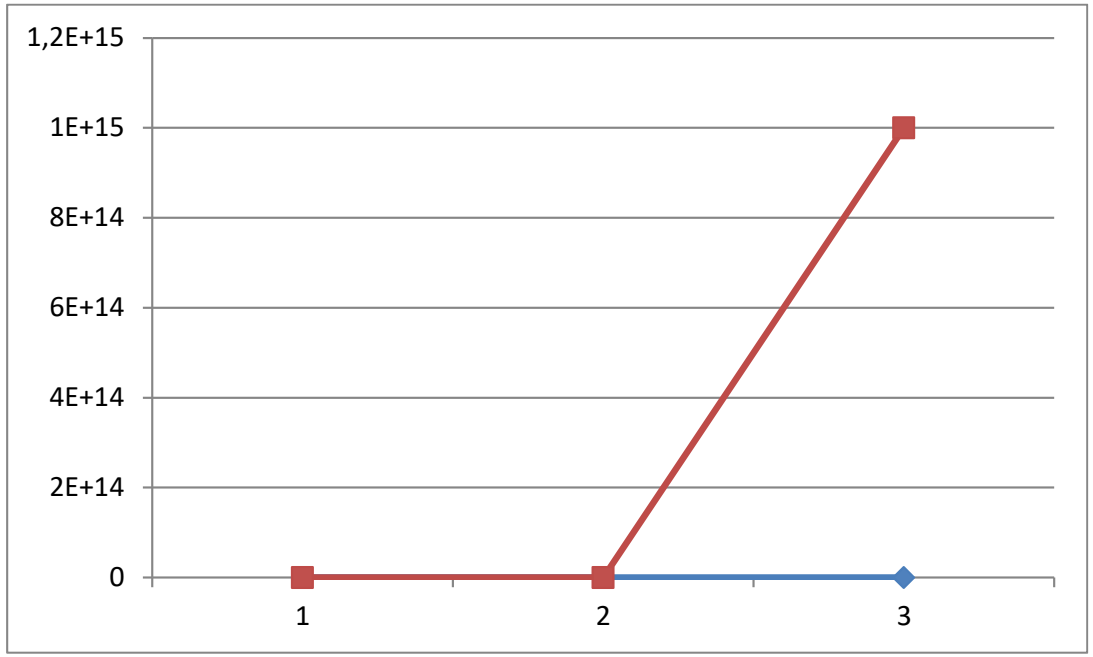
Tamanho da entrada X número de passos ?

Algoritmos Pseudopolinomiais

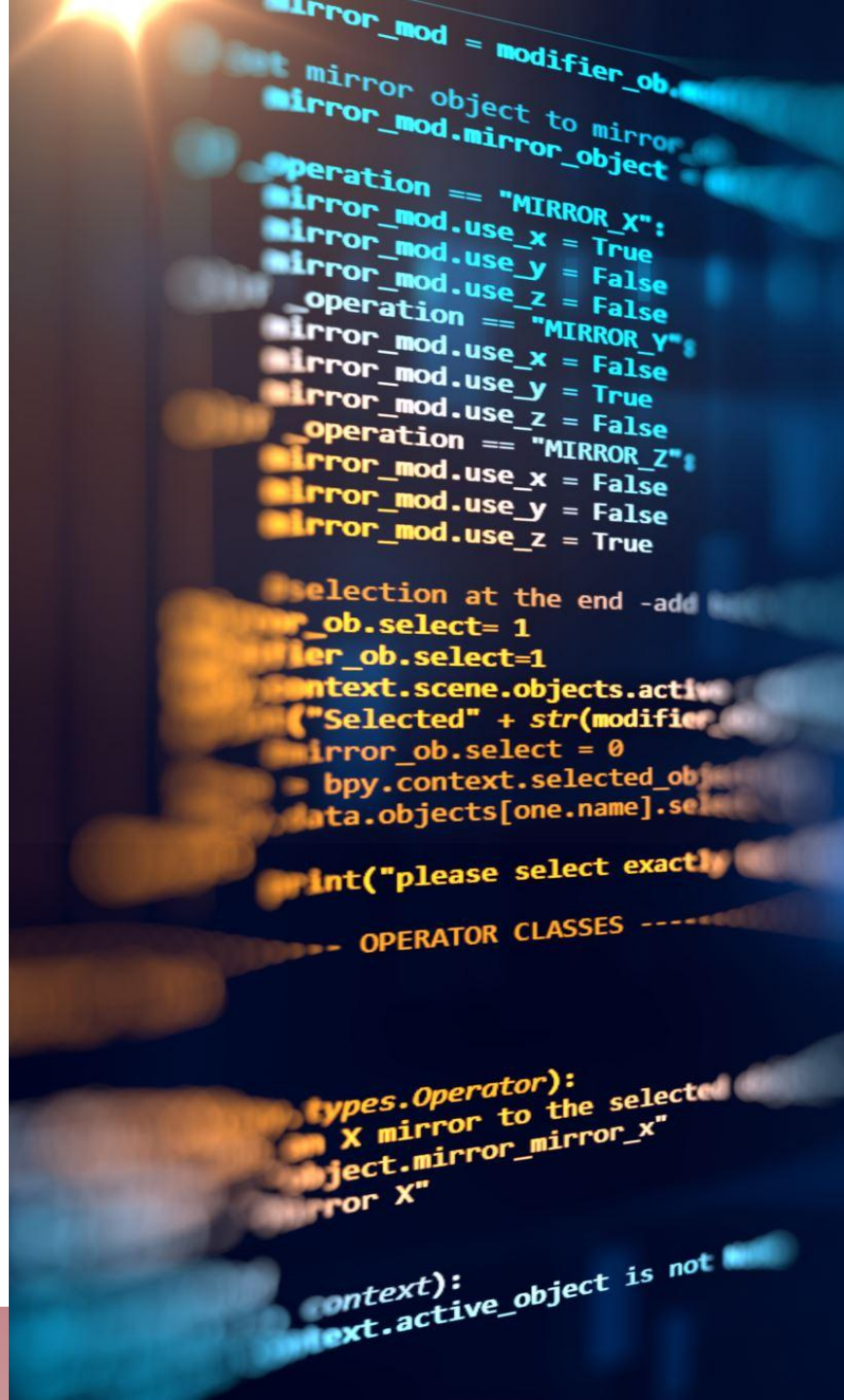
n	Log n	$(n)^{1/2}$
$1x10^{10}$	10	$1x10^{5}$
$1x10^{20}$	20	$1x10^{10}$
$1x10^{30}$	30	$1x10^{15}$

Tamanho da entrada codificada em binário

Número de passos



FIM


mirror_mod = modifier_ob.
mirror object to mirror
mirror_mod.mirror_object
operation == "MIRROR_X":
irror_mod.use_x = True
irror_mod.use_y = False
irror_mod.use_z = False
operation == "MIRROR_Y":
irror_mod.use_x = False
irror_mod.use_y = True
irror_mod.use_z = False
operation == "MIRROR_Z":
irror_mod.use_x = False
irror_mod.use_y = False
irror_mod.use_z = True
election at the end -add
ob.select= 1
er_ob.select=1
ntext.scene.objects.activ
("Selected" + str(modifier
irror_ob.select = 0
bpy.context.selected_obj
ata.objects[one.name].se
int("please select exactly
OPERATOR CLASSES
types.Operator):
X mirror to the selected
ject.mirror_mirror_x"
ror X"
context):
ext.active_object is not